

Q1. An in-order machine with 5 pipeline stages can issue, decode, and execute one instruction in a cycle. Assume memory/register reads and writes take a single cycle. There are separate instruction and data memories. Draw the pipeline diagram for the below instructions.

LW R2, 0(R5)
 LW R3, 0(R6)
 ADD R1, R2, R3
 SW R1, 0(R4)
 ADDI R4, R4, 4
 ADDI R5, R5, #4
 ADDI R6, R6, #4

- Assume that there is NO BYPASSING. Instruction stalls in the decode stage if any of the operands are not available.
- Assume that all bypass paths are provided.
- Assume bypassing and a pipeline where memory is composed of 2 stages each taking 1 cycle. Only one instruction can access the memory at a time. Note the memory stage would have a structural hazard only if an instruction is using the memory.

Ans.

a.

LW R2, 0(R5)	F	D	X	M	W														
LW R3, 0(R6)		F	D	X	M	W													
ADD R1, R2, R3			F	D	D	D	X	M											
SW R1, 0(R4)				F	F	F	D	D	D	X	M	W							
ADDI R4, R4, 4							F	F	F	D	X	M	W						
ADDI R5, R5, #4											F	D	X	M	W				
ADDI R6, R6, #4												F	D	X	M	W			

b. Assume that all bypass paths are provided.

LW R2, 0(R5)	F	D	X	M	W														
LW R3, 0(R6)		F	D	X	M	W													
ADD R1, R2, R3			F	D	D	X	M	W											
SW R1, 0(R4)				F	F	D	X	M	W										
ADDI R4, R4, 4						F	D	X	M	W									
ADDI R5, R5, #4							F	D	X	M	W								
ADDI R6, R6, #4								F	D	X	M	W							

c.

LW R2, 0(R5)	F	D	X	M1	M2	W									
LW R3, 0(R6)		F	D	X	X	M1	M2	W							
ADD R1, R2, R3			F	D	D	D	D	X	M1	M2	W				
SW R1, 0(R4)				F	F	F	F	D	X	M1	M2	W			
ADDI R4, R4, 4								F	D	X	M1	M2	W		
ADDI R5, R5, #4									F	D	X	M1	M2	W	
ADDI R6, R6, #4										F	D	X	M1	M2	W

Q2. Given the following 3-codes, calculate the number of mispredictions by 1-bit predictor. Assume Not Taken (NT) state at the start.

```
int sum = 0;
For (int i =0;i<50;i++){
    Sum +=i;
}
For (int i =0;i<50;i++){
    Sum +=i+50;
}
```

Ans.

For the first loop, the condition is given by:

MP = mispredict
CP = correct predict
→ For predictor with 1-bit history table
i=0 NT T MP
I =1 T T CP
I = 2 T T CP
...
I = 49 T T CP
I = 50 T NT MP

At the end of the first loop, NT state is present again. The second loop has the same mispredictions (2) as the first one. Total number of mispredictions = $2 * 2 = 4$

Q3. What states do 1-bit and 2-bit branch predictors go through for the following branch patterns?
History of the branch is N T N T N T for 1 bit, and 2 bit write the worst case prediction pattern for

least accuracy(mention accuracy also) in both the cases (Assume it starts from strongly not taken in both 1 bit and 2 bit)

History	N	T	N	T	N	T
1 bit						
Prediction	N					
Accuracy						
Next State						
2 bit						
Prediction	N					
Accuracy						
Next State						

Ans.

History	N	T	N	T	N	T
1-bit						
Prediction	N	N	T	N	T	N
Next State	0	1	0	1	0	1
Accuracy	1/7					
2 bit						
Prediction	N	N	N	N	N	N
Next State	00	01	00	01	00	01
Accuracy	4/7					

Q4. Schedule the following instructions using the VLIW EQ model. There are two integer units, one floating point unit and two load/store units. Floating point instructions are indicated using the “.S” extension. All floating point operations take 3 clock cycles, integer one and load-store take 4 clock cycles. Assume that the compiler does not reorder the instructions and branches are implemented using integer units.

```

loop:MUL R1, R2, R3
LW R2, 0(R3)
DIV.S F1, F2, F3
ADD R3, R1, R4
SUB R5, R6, R7
ADD.S F4, F5, F6
SW R5, 4(R7)
BNE R1, R5, loop

```

Ans. Note: Multiple Ways exist to solve. This is one of the ways.

I1	I2	F	LS1	LS2
MUL R1, R2, R3		DIV.S F1, F2, F3	LW R2, 0(R3)	
SUB R5, R6, R7	ADD R3, R1, R4	ADD.S F4, F5, F6	SW R5, 4(R7)	
BNE R1, R5, loop				

Note that there is no RAW hazard while scheduling MUL R1, R2, R3 and LW R2, 0(R3) as LW modifies R2 only at the end of 5th cycle. Hence, MUL reads the old value of R2.

Q5. Assume I2OI architecture with in-order fetch, decode and issue; out-of-order execute and writeback; in-order commit. The pipeline has 4 functional units: ALU (1 cycle X0), Loads (2 cycles, L0 and L1), Store (1 cycle, S0) and multiply (4 cycles, Y0, Y1, Y2, Y3).

```

0: MUL R6, R7, R8
1: ADD R9, R6, R11
2: MUL R7, R1, R2
3: LW R10, R12

```

Show the pipeline diagram.

Ans.

MUL R6, R7, R8	F	D	I	Y0	Y1	Y2	Y3	W	C						
ADD R9, R6, R11		F	D	I	I	I	I	X0	W	C					
MUL R7, R1, R2			F	D	D	D	D	I	Y0	Y1	Y2	Y3	W	C	
LW R10, R12				F	F	F	F	D	I	L0	L1	W	r		C

Q6. Consider a Prefetch-on-miss hardware data prefetching that prefetches 'n+1' when 'n' memory access is requested. The prefetched data is placed in the cache like any other memory access. When the prefetched data ('n+1') is requested, it prefetches the data in the next address ('n+2'). No prefetching is done if the data of the address is present already.

Given a empty 2 way set associative cache with eight 2-byte locations and with the following memory configuration, determine the data in cache with each memory access.

Access Pattern:100, 109, 108, 107, 101, 102, 103

Note that memory address are in decimal.

Address	Data
100	0x100
101	0x101
102	0x102
103	0x103
104	0x104
105	0x105
106	0x106
107	0x107
108	0x108
109	0x109
110	0x110

Ans. Given the cache with 8 locations and 2 way set associative, the number of sets are 4. Hence, the set is determined by (address mod 4) scheme. Given, memory access pattern as:

(Yellow: Set 0, Green:Set 1, Blue: Set 2, Red: Set 3)

100:

0x100
0x101

109:

0x100
0x101
0x109
0x110

108:

0x100
0x108
0x101
0x109
0x110

107:

0x100
0x108
0x101
0x109
0x110
0x107

0x101:

0x100
0x108
0x101
0x109
0x110
0x102
0x107

0x102:

0x100
0x108
0x101

0x109
0x110
0x102
0x107
0x103

Q7. Consider the code snippet in code below. Suppose that it is executed on a system with a 2-way set associative 16KB data cache with 32-byte blocks, 32-bit words, and an LRU replacement policy. Assume that int is word-sized. Also assume that the address of 'a' is 0x0, 'i' and 'x' are in registers, and that the cache is initially empty. How many data cache misses are there? How many hits are there?

```
int i;
int a[1024*1024];
int x=0;

for (i=0; i<1024; i++){
  x+= a[i] + a[1024*i];
}
```

Ans:

The number of sets in the cache = $(16 * 2^{10}) / (2 * 32) = 256$

Since a word size is 4 bytes, int is word sized and the size of a cache block is 32 bytes, the number of ints that would fit in a cache block is 8.

Therefore all the ints in a from a[0] to a[1023] map to the one of the cache lines of the sets 0 to 127, while all the ints in a from a[1024] to a[1024*2 -1] map to the sets 128 to 255. Similarly the array elements a[1024*2] to a[1024*3-1] map to cache lines of sets 0 to 127, a[1024*3] to a[1024*4 - 1] map to cache lines 128 to 255 and so on.

In the loop, every time a[i] is accessed for i being a multiple of 8 would be a miss. There the number of misses due to a[i] accesses inside the loop is $1024/8 = 128$. The number of hits is 896.

Now all accesses to a[1024*i] within the loop are misses. This is because map alternately to sets 0 and 128 consecutively where there are cold misses the first time they are referenced.

The total number of misses = $1024 + 128 = 1152$

The total number of hits = 896.

Q8. Assume that a branch in a code have the following outcomes.

T, NT, T, NT, T, NT

What is the accuracy of always-taken and always-not-taken predictors for this sequence of branch outcomes?

Ans:

Always Taken there are 3 / 6 (50%)correct predictions

Always Not-Taken there are 3/6 (50%) correct predictions

Q9. How many SRAM bits are needed to implement an 8KB two-way set associative cache with 64B block size? Assume that each line (entry) has a single valid bit and no dirty bits. There is one bit per set for true LRU. Assume that the address size of the machine is 32-bits and that the machine allows for byte addressing.

Ans:

Size of the cache = 8KB = 8×1024 B

Block Size = 64B

Total number of blocks = $(8 \times 1024) / 64 = 128$

total number of sets = $(8 \times 1024) / (64 \times 2) = 64$

Total number of SRAM bits = data bits + overload bit.

Overload bits = tag bits + valid bits + LRU bits

Total address bits = tag bits + index bits + offset bits

Index bits = $\log_2(\text{total no. of sets}) = \log_2(64) = 6$

Offset bits = $\log_2(\text{block size}) = \log_2(64) = 6$

Tagbits per block = address bits - (index bits + offset bits) = $32 - 12 = 20$

Total overload bits = $20 \times 128 + 1 \times 128 + 1 \times 64$
= 2752 bits = 344 Bytes

Total data bits = $8 \times 1024 \times 8$
= 65536 = 8192 Bytes

So, total number of SRAM bits = $65536 + 2752$
= 68,288 SRAM bits